

可以，下面我直接给你一版正式可归档的《LoveAdvisor V3 项目架构与实施方案》。这版是基于你现有 V1→V2.5 的演化逻辑继续往前推出来的：V1 解决可行性，V2 解决规则化，V2.5 解决评测工程化；V3 要解决的，是“脱离 Coze 依赖，做成真正本地可控、可持续迭代的系统”。这个方向也和你现有的项目背景、V2 标准架构、V2.5 评测体系是一致的。

LoveAdvisor V3 正式实施方案

本地可控化与产品化重构阶段

一、V3 阶段定义

1.1 为什么必须进入 V3

V2 和 V2.5 已经证明三件事：

第一，你的主业务逻辑是成立的。
第二，规则系统 + 评测系统已经形成了项目护城河。
第三，当前最大问题已经不再是“会不会做”，而是外部平台依赖过强。V2.5 虽然已经建立了评测驱动开发，但主链路仍依赖 Coze，这会带来额度、链路长度、调试受限、上下文不可控等问题。

所以 V3 的本质不是小修小补，而是一次架构主权回收。

1.2 V3 的阶段名称

建议正式命名为：

LoveAdvisor V3：本地可控化与产品化重构阶段

这个名字强调两件事：

-

本地可控化：核心链路从 Coze workflow 迁移到本地 Python + API 编排

-
-

产品化：系统从“工程原型”向“可维护产品”演进

-

1.3 V3 的核心目标

V3 的目标不再是继续优化某几个规则，而是完成四项升级：

目标编号	目标名称	核心内容
G1	主链路去 Coze 化	用本地 pipeline 替代 Coze workflow
G2	模块标准化	S2/S3/S5 改为可复用的本地 LLM 模块
G3	评测闭环内生	化 评测系统直接服务于本地 pipeline
G4	产品化底座搭建	为历史记录、配置化、版本化打基础

二、V3 的总体原则

2.1 原则一：模型调用只是能力，不是架构中心

- V1 时代，模型几乎等于系统。
- V2 时代，规则开始接管判断。
- V3 时代，模型应该只是一种可替换能力源。

也就是说，系统中心不再是 Coze 或某个工作流平台，而是：

本地编排层 + 明确协议 + 可替换模型接口

2.2 原则二：关键判断必须留在本地

V2 已经明确了项目的方法论核心：

规则主导决策（R1） + 模型负责表达（S5）

V3 继续坚持这一点，并进一步强化：

-

relationship_stage: 本地规则判定

-

-

interest_level: 本地规则判定

-

-

next_step 的边界: 本地约束

-

-

LLM: 只负责信号抽取与自然语言生成

-

2.3 原则三：节点减少，但职责更清楚

V1/V2 的链路偏“ workflow 节点化”。

V3 应该转为“服务模块化”。

也就是从：

节点思维

转向：

模块思维

不再强调 S1/S2/S3 是 workflow 中的一个块，而是强调：

-

preprocess_service

-

-

signal_extractor

-
-

rule_engine

-
-

strategy_generator

-
-

output_builder

-

2.4 原则四：系统必须可脱网调试、可局部替换

V3 必须支持三种运行模式：

-

在线真实模式：真实调用模型 API

-
-

离线回放模式：读取冻结的 S2/S3 输出做规则回归

-
-

Mock 模式：用伪造结果测前后端与展示层

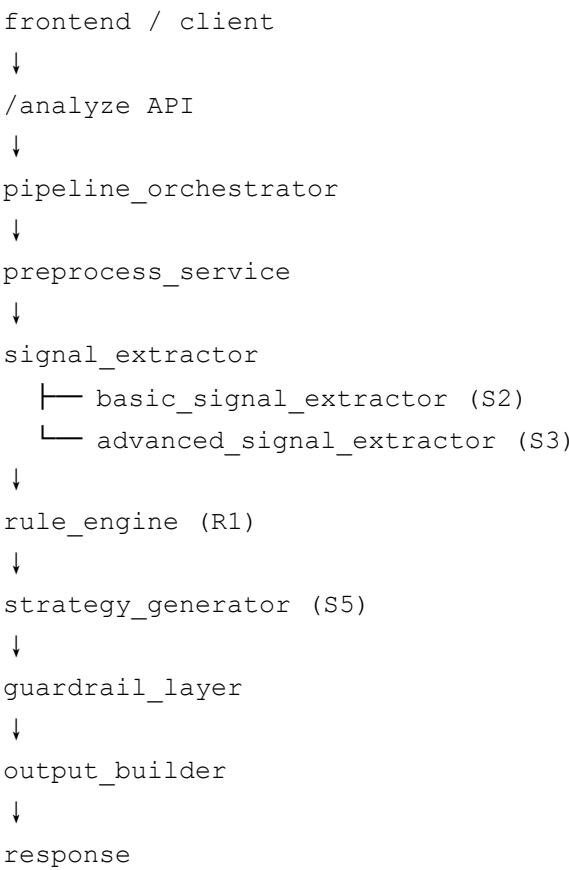
-

这会极大提升你的开发效率。

三、V3 总体架构设计

3.1 V3 主链路

V3 推荐架构如下：



这条链路本质上继承了 V2.5 的核心能力，但把 Coze 侧的逻辑收回到本地。现有 V2.5 的标准分层结构，也适合作为 V3 的出发点。

3.2 V3 相比 V2.5 的关键变化

模块	V2.5	V3
编排方式	Coze workflow + 后端拼接	本地 Python 编排
S2/S3	Coze 节点	本地 Prompt 模块 / API 调用
R1	本地规则	保留并增强

模块	V2.5	V3
S5	Coze 节点	本地生成模块
01	本地组装	保留
评测	旁路子系统	直接接入主系统
配置	workflow_id 驱动	model provider / prompt version 驱动

3.3 为什么 V3 要这样设计

因为你 V2.5 已经拥有：

- 标准工程目录
- •
规则层
- •
输出组装层
- •
自动评测子系统
-

所以 V3 不是推翻重做，而是：

保留成熟部分，替换受限部分

保留：

- rules.py
- •

output_builder.py

-
-

test_system

-
-

frontend 基本交互方式

-

替换：

-

coze_client.py

-
-

result_parser.py 中与 Coze 强绑定部分

-
-

workflow_id 配置体系

-

四、V3 标准项目架构

下面给你一版可直接作为标准目录的 V3 架构。

```
LoveAdvisorV3/
|
├── app/
|   ├── __init__.py
|   ├── main.py
|   |
|   └── api/
```

```
| |   ├── __init__.py
| |   ├── analyze.py
| |   ├── health.py
| |   └── debug.py
| |
| |   ├── core/
| |       ├── __init__.py
| |       ├── pipeline_orchestrator.py
| |       ├── runtime_context.py
| |       └── enums.py
| |
| |   ├── services/
| |       ├── __init__.py
| |       ├── preprocess_service.py
| |       ├── signal_extractor.py
| |       ├── strategy_generator.py
| |       ├── output_builder.py
| |       ├── guardrail_service.py
| |       └── history_service.py
| |
| |   ├── llm/
| |       ├── __init__.py
| |       ├── base_provider.py
| |       ├── provider_factory.py
| |       ├── openai_provider.py
| |       ├── deepseek_provider.py
| |       └── mock_provider.py
| |
| |   ├── prompts/
| |       ├── __init__.py
| |       ├── preprocess_prompt.py
| |       ├── s2_prompt.py
| |       ├── s3_prompt.py
| |       ├── s5_prompt.py
| |       └── prompt_versions.py
| |
| |   ├── parsers/
| |       ├── __init__.py
| |       ├── s2_parser.py
| |       ├── s3_parser.py
| |       └── s5_parser.py
| |
| |   ├── rules/
| |       ├── __init__.py
```



```
| | | | rules.py
| | | | stage_rules.py
| | | | interest_rules.py
| | | | next_step_policy.py
| | |
| | | | schemas/
| | | | | | __init__.py
| | | | | | request_models.py
| | | | | | signal_models.py
| | | | | | result_models.py
| | | | | | eval_models.py
| | | |
| | | | | | utils/
| | | | | | | | __init__.py
| | | | | | | | json_utils.py
| | | | | | | | logging_utils.py
| | | | | | | | retry_utils.py
| | | | | | | | text_utils.py
| | |
| | | | configs/
| | | | | | settings.py
| | | | | | model_registry.yaml
| | | | | | prompt_registry.yaml
| | | | | | env.example
| | |
| | | | frontend/
| | | | | | streamlit_app.py
| | | | | | components/
| | |
| | | | test_system/
| | | | | | data/
| | | | | | | | test_cases.json
| | | | | | | | frozen_s2_outputs.json
| | | | | | | | frozen_s3_outputs.json
| | | | | |
| | | | | | | | scripts/
| | | | | | | | | | batch_test.py
| | | | | | | | | | metrics_report.py
| | | | | | | | | | regression_test.py
| | | | | | | | | | llm_ab_test.py
| | | | | |
| | | | | | | | output/
| | | | | | | | | | test_results.csv
| | | | | | | | | | metrics_report.json
```

```
|       └── ab_test_report.json
|
|── data/
|   ├── history/
|   ├── cache/
|   └── snapshots/
|
|── docs/
|   ├── architecture/
|   ├── dev_logs/
|   ├── test_reports/
|   └── product_notes/
|
|── requirements.txt
|── README.md
|── run.py
```

这版结构是在你 V2.5 标准架构基础上升级出来的：保留分层思想，但把 Coze 绑定改成了通用 LLM Provider 机制。

五、V3 核心模块说明

5.1 pipeline_orchestrator

这是 V3 的新核心。

职责是：

-

接收一次分析请求

-

-

组织 preprocess → S2 → S3 → R1 → S5 → guardrail → O1

-

-

记录中间结果

-
-

统一异常处理

-
-

支持 debug 模式导出中间层输出

-

V2.5 的 `pipeline_service.py` 可以作为它的前身。

5.2 llm provider 层

这是 v3 最关键的新设计。

抽象接口示意：

```
class BaseLLMProvider:
    def generate_json(self, prompt: str, temperature: float = 0.2) -> dict:
        ...
```

好处是：

-

今天可以用 DeepSeek

-
-

明天可以换 OpenAI

-
-

测试时可以换 Mock

-
-

同一套业务代码不变

-

这一步会彻底摆脱 `workflow_id` 绑定。

5.3 prompts 层

V2 时代 `prompt` 很多是在 Coze 里。

V3 要把 `prompt` 资产正式代码化、版本化。

每个 `prompt` 都要有：

-

作用说明

-

-

输入字段

-

-

输出 schema

-

-

当前版本号

-

-

最近改动原因

-

这样你以后优化 `prompt`，不再是“黑箱修改”，而是“版本演化”。

5.4 parsers 层

模型输出永远不能直接信任。

V3 要在 prompt 后面配 parser:

- s2_parser: 兜底 initiative / response_length 等字段
- •
s3_parser: 兜底布尔信号与 signal_summary
- •
s5_parser: 兜底 suggestions / next_step
-

这会显著提升系统稳健性。

5.5 guardrail_service

这是 V3 很值得加的一层。

它不做主判断，但做最后一道边界控制：

- 如果 R1 = 无法判断，next_step 不允许推进
- •
如果 R1 = 冷淡期，建议不能写成“主动表白”
- •
如果 suggestions 偏离阶段，执行收敛修正

-

V2.5 已经明确“建议层必须服从规则层”，V3 应该把这个原则显式模块化。

5.6 history_service

V3 开始建议加入轻量历史记录能力：

-

保存请求文本

-

-

保存结果

-

-

保存中间层调试输出

-

-

保存使用的模型版本与 prompt 版本

-

这样你后面才能真正做产品化和错误复盘。

六、V3 数据协议设计

6.1 输入协议

```
{  
  "chat_text": "A: ...\nB: ...",  
}
```

```
"user_question": "她现在对我是什么态度？",
"debug": false,
"mode": "online"
}
```

6.2 S2 输出协议

```
{
  "initiative": "A 更主动",
  "response_length": "中",
  "emotional_tone": "温",
  "topic_depth": "中",
  "interaction_reciprocity": "正向承接",
  "key_signals": [
    "A 主动发起话题",
    "B 有承接但不算强烈"
  ]
}
```

这个协议继承自 V2 方案。

6.3 S3 输出协议

```
{
  "has_intimacy_signal": false,
  "has_relationship_probe": true,
  "has_positive_reciprocity": true,
  "has_rejection_signal": false,
  "has_push_pull_pattern": false,
  "has_sustained_coldness": false,
  "signal_summary": [
    "存在试探性关系话题",
    "对方给予正向承接"
  ]
}
```

这个协议也延续自 V2 的高级信号层设计。

6.4 R1 输出协议

```
{  
  "relationship_stage": "暧昧期",  
  "interest_level": "中"  
}
```

6.5 S5 输出协议

```
{  
  "psychological_analysis": "目前关系存在试探性靠近，但整体还没有进入非常稳定的高确认阶段。",  
  "risk_points": [  
    "若继续明显提速，容易让对方后撤",  
    "当前信号偏中等，不能过度解读"  
  ],  
  "suggestions": [  
    "保持轻松互动，不要突然上强度",  
    "优先延续对方愿意承接的话题",  
    "继续观察主动性是否能持续"  
  ],  
  "next_step": "先顺势承接互动，不要主动升级关系定义。"  
}
```

6.6 最终输出协议

```
{  
  "relationship_stage": "暧昧期",  
  "interest_level": "中",  
  "key_signals": [],  
  "signal_summary": [],  
  "psychological_analysis": "",  
  "risk_points": [],  
  "suggestions": [],  
  "next_step": "",  
  "meta": {  
    "model_provider": "deepseek",  
    "prompt_version": "v3.0.1",  
    "debug_id": "20260410_xxx"  
  }  
}
```


}

V3 建议正式加入 `meta` 字段，方便调试与版本追踪。

七、V3 开发重点

7.1 不是“继续补规则”，而是“重建可控主链路”

V2.5 的核心矛盾是测试工程化。

V3 的核心矛盾则是：

主链路可控性不够，系统主权还不在你自己手里

7.2 不是“平台 workflow 调参”，而是“本地服务系统化”

V1 适合用 Coze 打原型。

但到了 V3，再继续依赖平台节点已经不划算了。

因为你的项目已经具备：

-

清晰的业务边界

-

-

稳定的规则核

-

-

结构化输出协议

-

-

回归评测体系

-

这是非常典型的“该收回本地”的阶段。

八、V3 分阶段实施安排

下面给你一版 8 天执行方案。

Day 1: V3 架构冻结

目标：完成 V3 标准架构设计与迁移边界定义。

当天任务：

-

确认保留模块与替换模块

-

-

冻结 V3 目录结构

-

-

定义 provider 层接口

-

-

定义新配置项

-

-

确定 debug / online / mock 三种运行模式

-

产出：

-

V3 架构图

-

-

模块职责表

-

-

配置协议文档

-

Day 2: LLM Provider 层落地

目标：建立本地模型调用底座。

当天任务：

-

编写 `base_provider.py`

-

-

实现 `deepseek_provider.py`

-

-

实现 `mock_provider.py`

-

-

加入重试、超时、基础日志

-
-

跑通单次 JSON 输出测试

-

产出：

-

provider 层代码

-
-

统一调用接口

-
-

provider 测试脚本

-

Day 3: S2/S3 本地化迁移

目标：将 Coze 的信号提取能力迁移到本地。

当天任务：

-

将原 S2 prompt 转为本地 prompt 模块

-
-

将原 S3 prompt 转为本地 prompt 模块

-
-

编写 s2_parser / s3_parser

-
-

跑通单样本信号提取

-

产出：

-

s2_prompt.py / s3_prompt.py

-
-

s2_parser.py / s3_parser.py

-
-

单样本调试结果

-

Day 4: R1 接入与主链路打通

目标：完成 preprocess → S2 → S3 → R1 的本地主链路。

当天任务：

-

保留并迁移 rules.py

-
-

将原 infer 逻辑拆成 stage_rules / interest_rules

-

-

让 orchestrator 能直接读取 S2/S3 输出并产出 R1

-

产出:

-

rule_engine 模块

-

-

第一版本地主链路

-

Day 5: S5 与 guardrail 接入

目标: 完成建议层本地化, 并确保其服从规则。

当天任务:

-

将原 S5 prompt 转为本地 prompt 模块

-

-

编写 s5_parser

-

-

新增 guardrail_service

-

-

重点修正 next_step 的边界漂移

-

产出：

-

strategy_generator.py

-

-

guardrail_service.py

-

-

建议层收敛版

-

Day 6: 01 + API + 前端联调

目标：打通完整前后端链路。

当天任务：

-

完成 output_builder

-

-

修改 /analyze API

-

-

Streamlit 前端适配新字段

-

-

加入 debug 展示模式

-

产出:

-

可运行 V3 demo

-
-

前后端联调版本

-

Day 7: 评测系统适配 V3

目标: 让 V2.5 的 test_system 直接驱动 V3。

当天任务:

-

修改 batch_test.py 适配 V3 输出

-
-

增加 frozen_s2 / frozen_s3 回放模式

-
-

增加 provider A/B 测试能力

-
-

输出 V3 第一轮指标基线

-

产出：

-

V3 批测报告

-

-

provider 对比报告

-

-

回归结果表

-

Day 8：阶段验收与文档归档

目标：完成 V3 第一轮版本归档。

当天任务：

-

输出 V3 架构文档

-

-

输出 V3 Prompt 版本清单

-

-

输出 V3 基线测试报告

-

-

决定下一步进入“精修”还是“产品化”

-

产出：

-

V3 正式归档文件

-

-

阶段验收结论

-

九、V3 验收标准

如果满足以下条件，可以认为 V3 第一阶段成功：

-

主链路不再依赖 Coze

-

-

S2/S3/S5 可以本地调用模型 API

-

-

R1 保持本地规则主导

-

-

test_system 能直接评测 V3

-

-

debug 模式可导出中间层结果

-
-

next_step 不突破阶段边界

-
-

项目可以在本地持续迭代，不再受平台额度卡死

-

十、V3 的预期收益

维度	V2.5	V3 预期
可控性	中	高
调试效率	中	高
平台依赖	高	低
评测闭环	已建立	更紧密
模型替换成本	高	低
产品化能力	初级	中级起步

十一、V3 之后的方向

V3 完成后，项目才真正具备继续往下走的资格。
之后可以分成两条线：

产品线

-

历史记录

-
-

用户画像

-
-

多轮对话分析

-
-

UI 卡片化

-

工程线

-

模型 A/B 测试

-
-

Prompt 版本管理

-
-

错误样本回放

-
-

缓存与性能优化

-

十二、最终结论

一句话总结 V3:

V3 不是继续做一个更复杂的工作流, 而是把 LoveAdvisor 从“平台上的项目”变成“你自己的系统”。

这一步和你前面整合出来的项目背景是完全连续的：

V1 是原型，V2 是规则化，V2.5 是评测工程化，**V3 就是系统主权回收 + 本地产品化底座搭建。**

如果你愿意，我下一步可以直接继续给你：

1) V3 标准项目文件架构 txt 版，或者 2) V3 第一天到第三天的可执行开发清单。